LEMBAR KERJA PRAKTIKUM (LKP)

MODUL 6

Perulangan Bersarang, Pola, Tabel, dan Debugging Program

Identitas	Isian
Nama	Fransiskus Reikinata Koesprasetyo
NIM	2610511036
Kelas	B
Tanggal Praktikum	30 September 2026

Petunjuk penggunaan LKP: Kerjakan kegiatan sesuai urutan pada Modul 6. Ketik program pada editor, kemudian lampirkan screenshot source code, hasil kompilasi, dan output pada tempat yang disediakan. Lengkapi tracing, analisis, eksperimen, debugging, dan pengujian sesuai instruksi.

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Membentuk Pola Segitiga

Gunakan program latihan_01.c pada Modul 6. Setelah mengetik program dan menjalankannya, dokumentasikan proses berikut.

Screenshot source code latihan_01.c

```c
#include <stdio.h>

int main(void)
{
    int tinggi;
    int baris;
    int kolom;

    do
    {
        printf("Masukkan tinggi pola 1-20: ");
        scanf("%d", &tinggi);

        if ((tinggi < 1) || (tinggi > 20))
        {
            printf("Tinggi harus berada pada rentang 1-20.\n");
        }
    } while ((tinggi < 1) || (tinggi > 20));

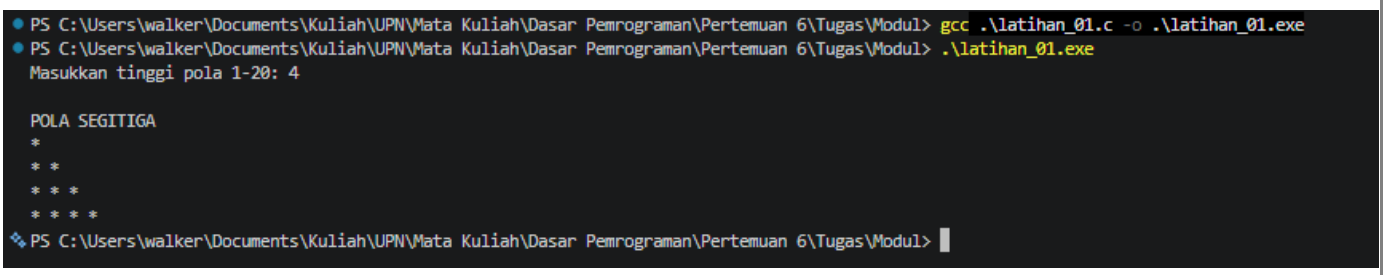
    printf("\nPOLA SEGITIGA\n");

    for (baris = 1; baris <= tinggi; baris++)
    {
        for (kolom = 1; kolom <= baris; kolom++)
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}
```

Screenshot output program (gunakan salah satu input valid)



Pertanyaan analisis

1. Apa fungsi perulangan luar?

Jawaban: Perulangan luar berfungsi untuk menentukan jumlah baris pada pola.

2. Apa fungsi perulangan dalam?

Jawaban: Perulangan dalam berfungsi untuk menentukan jumlah simbol yang ditampilkan pada setiap baris.

3. Mengapa kondisi perulangan dalam menggunakan kolom <= baris?

Jawaban: Karena jumlah simbol pada setiap baris mengikuti nomor baris. Baris 1 memiliki 1 simbol, baris 2 memiliki 2 simbol, dst.

4. Berapa kali printf("* ") dijalankan untuk tinggi 4?

Jawaban: 10 kali karena 1+2+3+4 = 10

5. Apa hasil jika menggunakan kolom < baris?

Jawaban: Jumlah simbol setiap baris menjadi berkurang satu. Baris pertama tidak menampilkan simbol.

6. Apa hasil jika printf("\n") dipindahkan ke dalam perulangan kolom?

Jawaban: Setiap simbol akan berpindah ke baris baru sehingga pola segitiga tidak terbentuk.

7. Apa hasil jika counter kolom tidak diinisialisasi kembali?

Jawaban: Perulangan dalam tidak berjalan dengan benar pada baris berikutnya karena nilai kolom masih berasal dari iterasi sebelumnya.

8. Mengapa tinggi dibatasi maksimal 20?

Jawaban: Agar ukuran pola tetap terkendali dan output tidak terlalu besar. Modul menetapkan batas tinggi 1–20.

2. Praktikum Terbimbing 2 — Membuat Tabel Perkalian

Gunakan program latihan_02.c pada Modul 6. Jalankan program dengan minimal satu batas yang valid, kemudian dokumentasikan hasilnya.

Screenshot source code latihan_02.c

```c
#include <stdio.h>

int main(void)
{
    int batas;
    int baris;
    int kolom;

    do
    {
        printf("Masukkan batas tabel 1-12: ");
        scanf("%d", &batas);

        if ((batas < 1) || (batas > 12))
        {
            printf("Batas harus berada pada rentang 1-12.\n");
        }
    } while ((batas < 1) || (batas > 12));

    printf("\nTABEL PERKALIAN\n\n");

    printf("%5s", "x");

    for (kolom = 1; kolom <= batas; kolom++)
    {
        printf("%5d", kolom);
    }

    printf("\n");

    for (baris = 1; baris <= batas; baris++)
    {
        printf("%5d", baris);

        for (kolom = 1; kolom <= batas; kolom++)
        {
            printf("%5d", baris * kolom);
        }

        printf("\n");
    }

    return 0;
}
```

Screenshot output tabel perkalian

```
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemuan 6\Tugas\Modul> .\latihan_02.exe

    x    1    2    3    4
    1    1    2    3    4
    2    2    4    6    8
    3    3    6    9   12
    4    4    8   12   16
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemuan 6\Tugas\Modul>
```

Pertanyaan analisis

9. Mengapa bagian header kolom dibuat menggunakan perulangan terpisah?

Jawaban: Karena header digunakan untuk menampilkan nomor kolom sebelum isi tabel dimulai.

10. Apa fungsi %5d?

Jawaban: Untuk memberikan lebar minimal 5 karakter sehingga angka pada tabel tersusun lebih rapi.

11. Berapa kali perkalian dilakukan untuk batas 4?

Jawaban: 16 kali

12. Berapa kali perkalian dilakukan untuk batas 10?

Jawaban: 100 kali

13. Mengapa nilai kolom dimulai kembali dari 1 pada setiap baris?

Jawaban: Karena setiap baris harus menghitung perkalian dari kolom pertama sampai kolom terakhir.

14. Apa akibatnya apabila printf("\n") dihapus?

Jawaban: Semua hasil tabel akan tersambung dalam satu baris sehingga bentuk tabel tidak rapi.

15. Bagaimana cara menampilkan tabel dari 0 sampai batas?

Jawaban: Counter baris dan kolom dimulai dari 0, kemudian perulangannya berjalan sampai nilai batas.

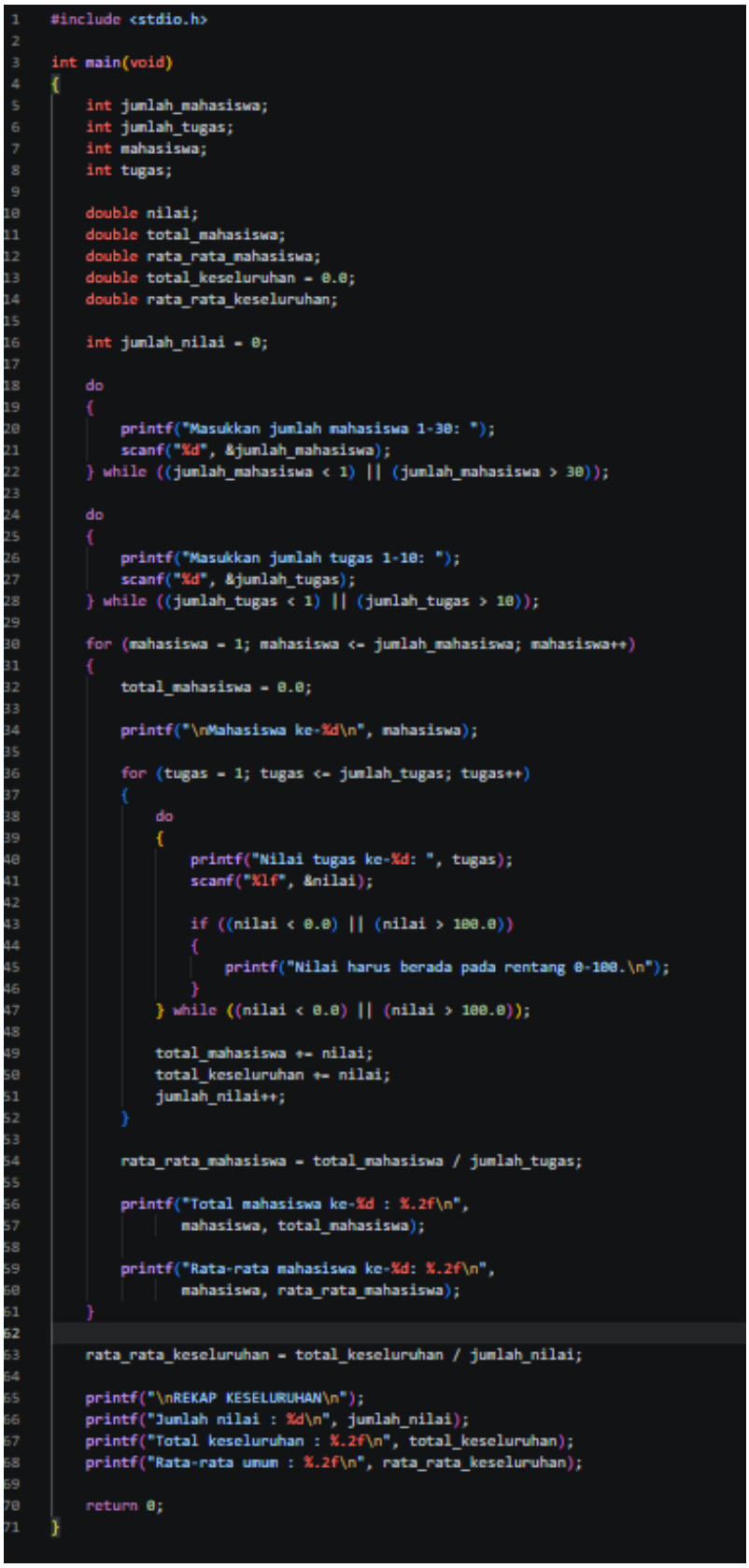
16. Apakah tabel dapat dibuat menggunakan while?

Jawaban: Ya. Nested loop tetap bisa dibuat menggunakan while selama inisialisasi, kondisi, dan update counter dibuat dengan benar.

3. Praktikum Terbimbing 3 — Rekap Nilai Beberapa Mahasiswa

Gunakan program latihan_03.c pada Modul 6. Uji dengan lebih dari satu mahasiswa dan lebih dari satu tugas agar nested loop terlihat jelas.

Screenshot source code latihan_03.c



```c
#include <stdio.h>

int main(void)
{
    int jumlah_mahasiswa;
    int jumlah_tugas;
    int mahasiswa;
    int tugas;

    double nilai;
    double total_mahasiswa;
    double rata_rata_mahasiswa;
    double total_keseluruhan = 0.0;
    double rata_rata_keseluruhan;

    int jumlah_nilai = 0;

    do
    {
        printf("Masukkan jumlah mahasiswa 1-30: ");
        scanf("%d", &jumlah_mahasiswa);
    } while ((jumlah_mahasiswa < 1) || (jumlah_mahasiswa > 30));

    do
    {
        printf("Masukkan jumlah tugas 1-10: ");
        scanf("%d", &jumlah_tugas);
    } while ((jumlah_tugas < 1) || (jumlah_tugas > 10));

    for (mahasiswa = 1; mahasiswa <= jumlah_mahasiswa; mahasiswa++)
    {
        total_mahasiswa = 0.0;

        printf("\nMahasiswa ke-%d\n", mahasiswa);

        for (tugas = 1; tugas <= jumlah_tugas; tugas++)
        {
            do
            {
                printf("Nilai tugas ke-%d: ", tugas);
                scanf("%lf", &nilai);

                if ((nilai < 0.0) || (nilai > 100.0))
                {
                    printf("Nilai harus berada pada rentang 0-100.\n");
                }
            } while ((nilai < 0.0) || (nilai > 100.0));

            total_mahasiswa += nilai;
            total_keseluruhan += nilai;
            jumlah_nilai++;
        }

        rata_rata_mahasiswa = total_mahasiswa / jumlah_tugas;

        printf("Total mahasiswa ke-%d : %.2f\n",
               mahasiswa, total_mahasiswa);

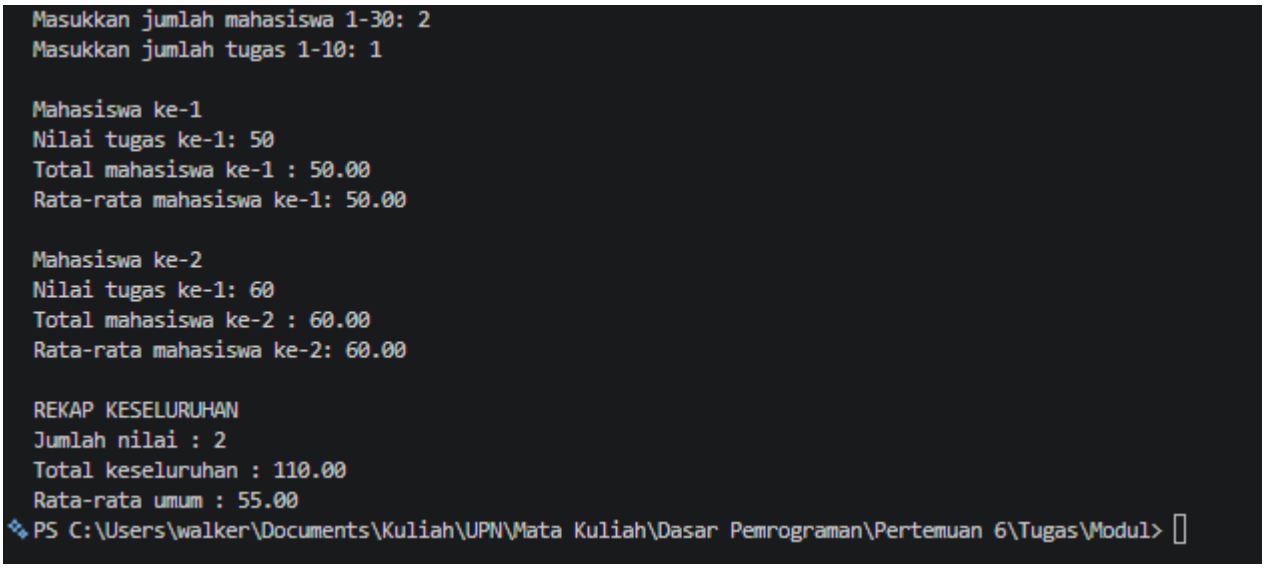
        printf("Rata-rata mahasiswa ke-%d: %.2f\n",
               mahasiswa, rata_rata_mahasiswa);
    }

    rata_rata_keseluruhan = total_keseluruhan / jumlah_nilai;

    printf("\nREKAP KESELURUHAN\n");
    printf("Jumlah nilai : %d\n", jumlah_nilai);
    printf("Total keseluruhan : %.2f\n", total_keseluruhan);
    printf("Rata-rata umum : %.2f\n", rata_rata_keseluruhan);

    return 0;
}
```

Screenshot output rekap nilai beberapa mahasiswa



Pertanyaan analisis

17. Mengapa total_mahasiswa diinisialisasi di dalam perulangan mahasiswa?

Jawaban: Supaya setiap mahasiswa memulai perhitungan total dari 0 dan tidak membawa nilai mahasiswa sebelumnya.

18. Mengapa total_keseluruhan diinisialisasi sebelum perulangan mahasiswa?

Jawaban: Karena variabel tersebut digunakan untuk menampung nilai dari seluruh mahasiswa sehingga tidak boleh direset setiap mahasiswa berganti.

19. Berapa jumlah input nilai apabila ada 4 mahasiswa dan 3 tugas?

Jawaban: 12 Nilai

20. Apa fungsi jumlah_nilai?

Jawaban: Untuk menghitung jumlah seluruh nilai yang dimasukkan dan digunakan sebagai pembagi untuk menghitung rata-rata umum.

21. Apakah jumlah_nilai dapat dihitung melalui perkalian?

Jawaban: Ya, jika setiap mahasiswa memiliki jumlah tugas yang sama:

22. Apa akibatnya apabila total_mahasiswa tidak diatur kembali menjadi 0?

Jawaban: Nilai mahasiswa sebelumnya akan ikut terbawa sehingga total dan rata-rata mahasiswa berikutnya menjadi salah.

23. Mengapa validasi nilai menggunakan do-while?

Jawaban: Supaya input nilai pasti diminta minimal satu kali dan akan diminta ulang apabila nilainya berada di luar rentang 0–100.

24. Bagaimana cara menentukan mahasiswa dengan rata-rata tertinggi tanpa menggunakan array?

Jawaban: Gunakan satu variabel untuk menyimpan rata-rata tertinggi sementara dan satu variabel untuk menyimpan nomor mahasiswa. Setiap rata-rata mahasiswa dibandingkan dengan nilai tertinggi tersebut.

4. Praktikum Terbimbing 4 — Menentukan Posisi pada Tabel

Gunakan program latihan_04.c pada Modul 6. Uji salah satu posisi target valid pada papan 5 x 5.

Screenshot source code latihan_04.c

```c
#include <stdio.h>

int main(void)
{
    int baris_target;
    int kolom_target;
    int baris;
    int kolom;

    do
    {
        printf("Masukkan baris target 1-5: ");
        scanf("%d", &baris_target);
    } while ((baris_target < 1) || (baris_target > 5));

    do
    {
        printf("Masukkan kolom target 1-5: ");
        scanf("%d", &kolom_target);
    } while ((kolom_target < 1) || (kolom_target > 5));

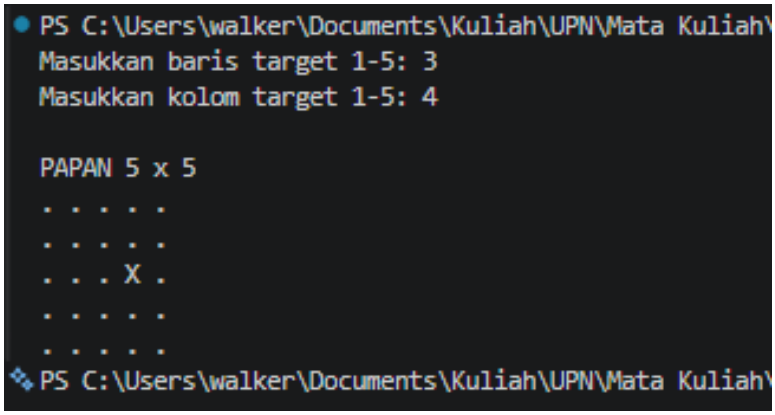
    printf("\nPAPAN 5 x 5\n");

    for (baris = 1; baris <= 5; baris++)
    {
        for (kolom = 1; kolom <= 5; kolom++)
        {
            if ((baris == baris_target) && (kolom == kolom_target))
            {
                printf("X ");
            }
            else
            {
                printf(". ");
            }
        }

        printf("\n");
    }

    return 0;
}
```

Screenshot output papan 5 x 5



Pertanyaan analisis

25. Mengapa digunakan operator &&?

Jawaban: Karena posisi X hanya boleh muncul jika baris **dan** kolom sama-sama sesuai dengan posisi target.

26. Apa yang terjadi apabila operator diganti menjadi ||?

Jawaban: X akan muncul pada banyak posisi karena cukup salah satu kondisi yang benar.

27. Berapa kali kondisi diperiksa?

Jawaban: 25 Kali

28. Apakah perulangan dapat dihentikan setelah X ditampilkan?

Jawaban: Bisa menggunakan break, tetapi break hanya menghentikan loop terdekat. Untuk program ini lebih baik seluruh papan tetap diproses.

29. Mengapa tetap diperlukan perulangan sampai seluruh papan selesai?

Jawaban: Supaya seluruh posisi papan mendapatkan output berupa X atau .

30. Bagaimana menampilkan diagonal utama menggunakan kondisi baris == kolom?

Jawaban: pakai if (baris == kolom) jika true maka akan tampil karakter diagonal

31. Bagaimana menampilkan bingkai papan?

Jawaban: menggunakan if (baris == 1 || baris == 5 || kolom == 1 || kolom == 5)

V. Eksperimen dan Tracing

1. Latihan Tracing

Gunakan program tracing.c pada Modul 6. Jangan jalankan program sebelum tabel tracing selesai.

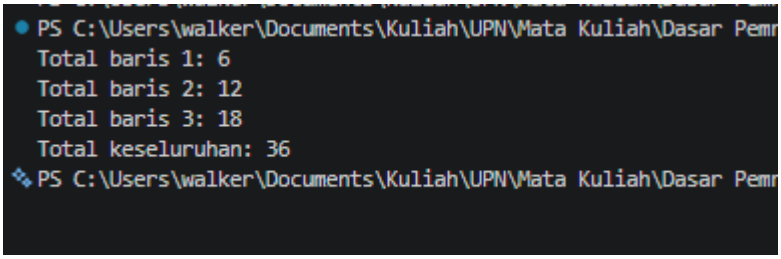
Baris	Kolom	baris × kolom	total_baris	total_keseluruhan
1	1	1	1	0
1	2	2	3	0
1	3	3	6	0
Setelah baris 1	–	–	6	6
2	1	2	2	6
2	2	4	6	6
2	3	6	12	6
Setelah baris 2	–	–	12	18
3	1	3	3	18
3	2	6	9	18

Baris	Kolom	baris × kolom	total_baris	total_keseluruhan
3	3	9	18	18
Setelah baris 3	–	–	18	36

Prediksi total setiap baris dan total keseluruhan

Baris 1 prediksi total 6 (1×1 + 1×2 + 1×3)
Baris 2 prediksi total 12 (2×1 + 2×2 + 2×3)
Baris 3 prediksi total 18 (3×1 + 3×2 + 3×3)
Keseluruhan total 36 (6 + 12 + 18)

Screenshot hasil eksekusi setelah tracing selesai



Jelaskan mengapa total_baris harus diatur menjadi nol pada awal setiap baris

Karena total_baris digunakan untuk menghitung total pada satu baris saja. Setelah selesai satu baris, nilainya harus dikembalikan ke 0 agar perhitungan baris berikutnya tidak ikut membawa hasil dari baris sebelumnya.

2. Eksperimen

No.	Perubahan	Prediksi	Aman dijalankan?	Hasil Aktual / Penjelasan
1	Pindahkan printf("\n") ke dalam loop kolom	Setiap simbol akan berpindah ke baris baru sehingga bentuk segitiga tidak terbentuk.	Ya	Setiap * tampil pada baris yang berbeda karena printf("\n") dijalankan setiap perulangan kolom.
2	Ubah batas baris menjadi 4	Program hanya akan mencetak 4 baris.	Ya	Program mencetak 4 baris sesuai batas perulangan baris yang diubah menjadi 4.
3	Ubah batas kolom menjadi kolom <= baris	Jumlah simbol pada setiap baris akan mengikuti nomor baris	Ya	Terbentuk segitiga, baris pertama 1 simbol, baris kedua 2 simbol, dan seterusnya.
4	Ubah batas kolom menjadi kolom < baris	Setiap baris akan memiliki satu simbol lebih sedikit dibanding sebelumnya.	Ya	Baris pertama tidak memiliki simbol, baris kedua memiliki 1 simbol, baris ketiga 2 simbol, dan seterusnya.
5	Gunakan baris * kolom sebagai output	Program akan	Ya	Pada setiap baris,

No.	Perubahan	Prediksi	Aman dijalankan?	Hasil Aktual / Penjelasan
		menampilkan hasil perkalian antara nomor baris dan kolom.		perulangan kolom berhenti saat kolom mencapai 3 sehingga kolom setelahnya tidak diproses.
6	Tambahkan break ketika kolom == 3	Perulangan kolom akan berhenti ketika mencapai kolom ke-3.	Ya	Pada setiap baris, perulangan kolom berhenti saat kolom mencapai 3 sehingga kolom setelahnya tidak diproses.
7	Tambahkan continue ketika kolom == 2	Kolom ke-2 akan dilewati tanpa menjalankan perintah setelah continue.	Ya	Saat kolom bernilai 2, output untuk kolom tersebut dilewati, kemudian perulangan lanjut ke kolom berikutnya.
8	Hapus inisialisasi kolom = 1 dari loop dalam	Nilai kolom tidak dimulai dari 1 sehingga hasil perulangan menjadi tidak sesuai.	Tidak	Program dapat menghasilkan output yang salah karena variabel kolom tidak memiliki nilai awal yang sesuai sebelum digunakan dalam loop.
9	Ubah update kolom menjadi kolom--	Nilai kolom akan terus berkurang sehingga kondisi perulangan dapat terus terpenuhi.	Tidak	Perulangan dapat menjadi infinite loop karena kolom terus berkurang dan tidak mencapai batas akhir yang ditentukan.
10	Tambahkan kondisi untuk menampilkan diagonal	Program hanya akan menampilkan simbol pada posisi baris dan kolom yang sama.	Ya	Simbol hanya muncul pada posisi diagonal, yaitu ketika baris == kolom.

Catatan: sebelum menjalankan setiap eksperimen, periksa kemungkinan infinite loop. Setelah selesai, kembalikan program ke bentuk awal sebelum melakukan eksperimen berikutnya.

Screenshot eksperimen yang paling menunjukkan perubahan perilaku program

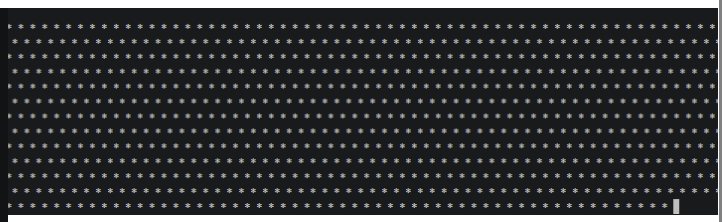
```c
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris <= 3; baris++)
    {
        for (kolom = 1; kolom <= 4; kolom--)
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}
```



VI. Debugging Challenge

1. Debugging Challenge 1 — Pola yang Tidak Sesuai

Program pada modul seharusnya menghasilkan pola segitiga 1 sampai 5 simbol. Jangan langsung menjalankan program sebelum memeriksa risiko infinite loop.

Screenshot source code awal (sebelum diperbaiki)

```c
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris < 5; baris++)
    {
        for (kolom = 1; kolom <= 5; kolom--)
        {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

No.	Bagian Bermasalah	Jenis Kesalahan	Penyebab	Perbaikan
1	baris < 5	Kurang tanda	Hasilnya hanya 1-4 tidak 1-5	Baris <= 5
2	kolom--	Infinite loop	Tidak ada batas paling bawah dan batas terkecil	Kolom++
3	kolom <= 5	Logical error	Setiap baris menghasilkan 5 simbol	Kolom <= baris

Screenshot source code setelah diperbaiki

```c
#include <stdio.h>

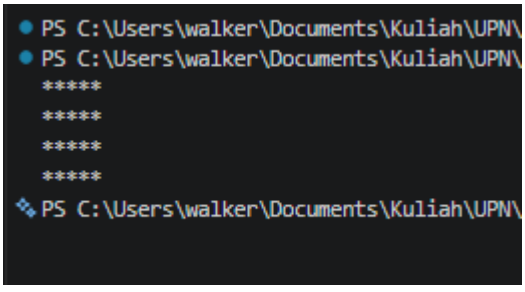
int main(void)
{
    int baris;
    int kolom;

    for (baris = 1; baris < 5; baris++)
    {
        for (kolom = 1; kolom <= 5; kolom++)
        {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

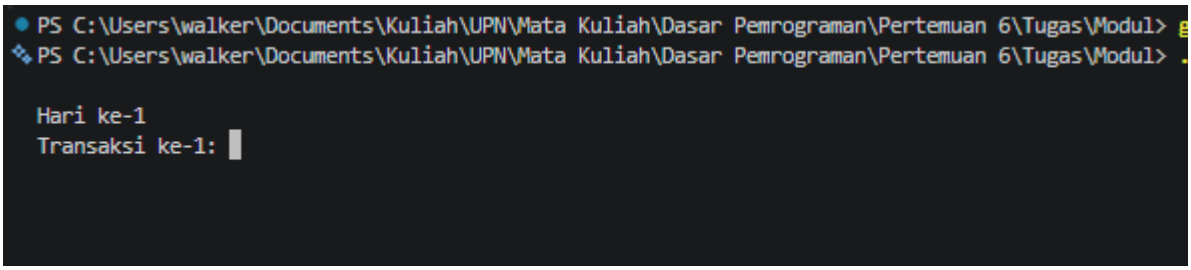
Screenshot output akhir yang sesuai pola



2. Debugging Challenge 2 — Rekap Penjualan yang Salah

Program dimaksudkan untuk mencatat penjualan 3 hari, masing-masing 4 transaksi. Kompilasi dengan warning aktif, perbaiki satu kesalahan pada satu waktu, lalu uji dengan data pada modul.

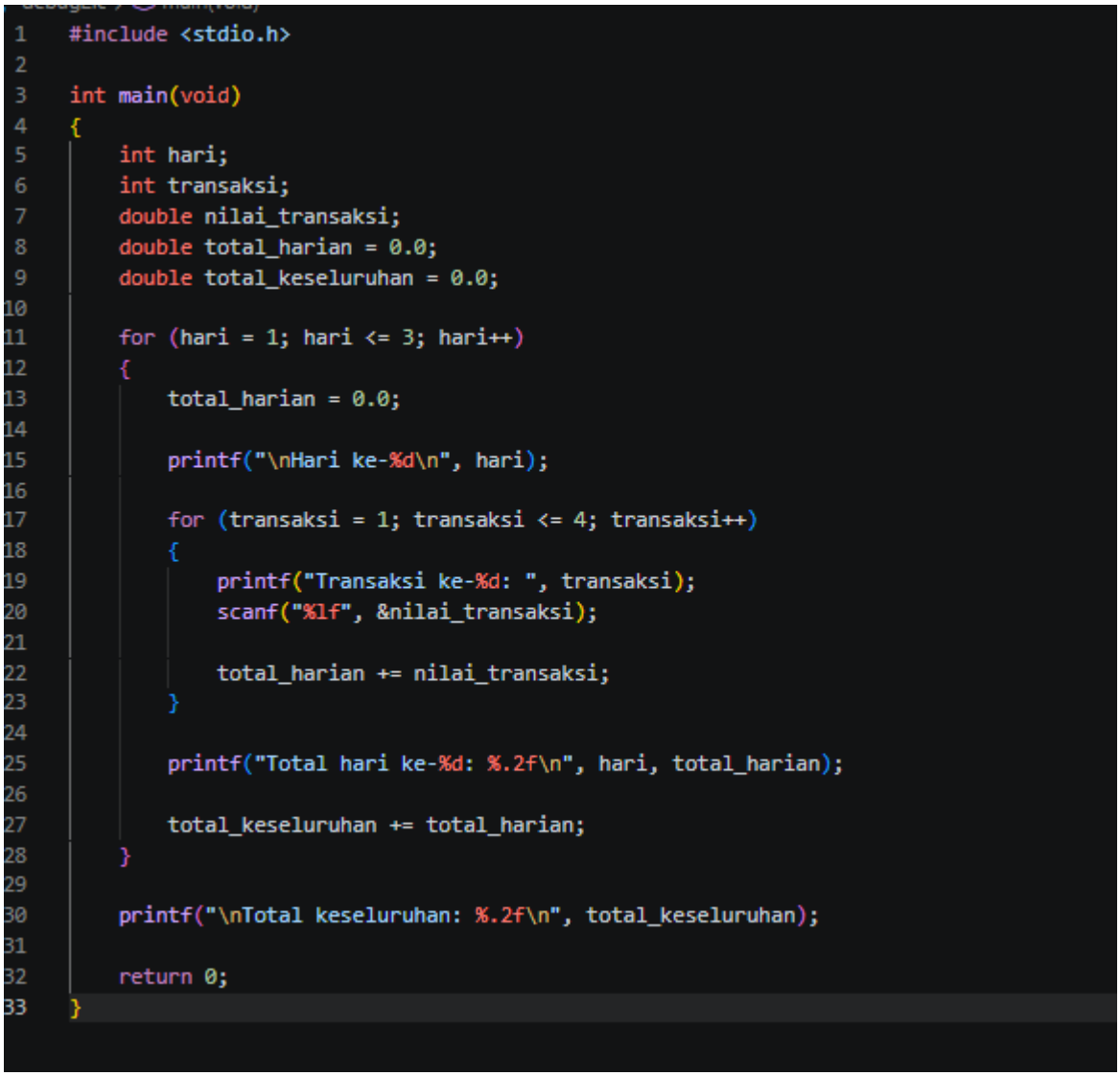
Screenshot kompilasi awal / warning



No.	Bagian Bermasalah	Error/Warning	Penyebab	Perbaikan
1	total_keseluruhan belum diinisialisasi	Hasil total keseluruhan tidak menentu.	Variabel belum memiliki nilai awal.	Beri nilai awal total_keseluruhan = 0.0.
2	total_harian tidak di-reset setiap hari	Total hari berikutnya ikut membawa nilai dari hari sebelumnya.	total_harian tidak dikembalikan ke 0 setiap pergantian hari.	Tambahkan total_harian = 0.0 di awal perulangan hari.
3	transaksi < 4	Hanya 3 transaksi yang diproses.	Kondisi transaksi < 4 berhenti saat transaksi bernilai 4.	Ubah menjadi transaksi <= 4.
4	scanf untuk nilai_transaksi	Data dapat salah dibaca jika variabel bertipe double tetapi format input tidak sesuai.	Format %d digunakan untuk variabel double.	Gunakan scanf("%lf", &nilai_transaksi).
5	total_harian = nilai_transaksi	Total hanya menyimpan nilai transaksi terakhir.	Nilai sebelumnya diganti, bukan ditambahkan.	Ubah menjadi total_harian += nilai_transaksi.
6	Nilai transaksi negatif	Program tetap menerima nilai transaksi negatif.	Tidak ada validasi untuk memastikan transaksi tidak negatif.	Gunakan do-while agar nilai transaksi harus >= 0.

Hari	Data Uji	Expected Total	Actual Total	Status
1	10, 20, 30, 40	100	100	Berhasil
2	5, 5, 5, 5	20	20	Berhasil
3	100, 0, 50, 50	200	200	Berhasil
Keseluruhan	-	320	320	Berhasil

Screenshot source code setelah diperbaiki



```c
#include <stdio.h>

int main(void)
{
    int hari;
    int transaksi;
    double nilai_transaksi;
    double total_harian = 0.0;
    double total_keseluruhan = 0.0;

    for (hari = 1; hari <= 3; hari++)
    {
        total_harian = 0.0;

        printf("\nHari ke-%d\n", hari);

        for (transaksi = 1; transaksi <= 4; transaksi++)
        {
            printf("Transaksi ke-%d: ", transaksi);
            scanf("%lf", &nilai_transaksi);

            total_harian += nilai_transaksi;
        }

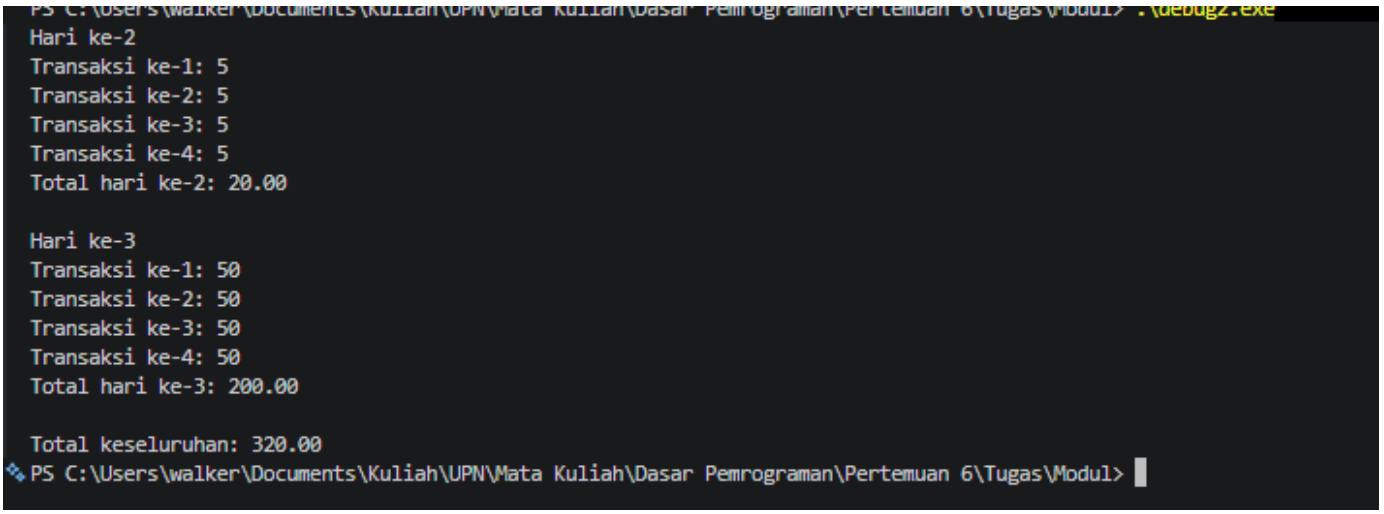
        printf("Total hari ke-%d: %.2f\n", hari, total_harian);

        total_keseluruhan += total_harian;
    }

    printf("\nTotal keseluruhan: %.2f\n", total_keseluruhan);

    return 0;
}
```

Screenshot output akhir



VII. Latihan Mandiri

1. Level 1 — Persegi Panjang Karakter

Buat program yang menerima jumlah baris, jumlah kolom, dan satu karakter. Baris dan kolom berada pada rentang 1–20, divalidasi menggunakan do-while, dan pola dibuat dengan nested for.

Screenshot source code

```c
#include <stdio.h>

int main(void)
{
    int baris;
    int kolom;
    int i;
    int j;
    char karakter;

    do
    {
        printf("Masukkan jumlah baris : ");
        scanf("%d", &baris);
    } while (baris < 1 || baris > 20);

    do
    {
        printf("Masukkan jumlah kolom : ");
        scanf("%d", &kolom);
    } while (kolom < 1 || kolom > 20);

    printf("Masukkan karakter : ");
    scanf(" %c", &karakter);

    for (i = 1; i <= baris; i++)
    {
        for (j = 1; j <= kolom; j++)
        {
            printf("%c ", karakter);
        }
        printf("\n");
    }

    return 0;
}
```

Screenshot output

```
Masukkan jumlah baris : 3
Masukkan jumlah kolom : 5
Masukkan karakter : #
# # # # #
# # # # #
# # # # #
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Ku
```

2. Level 2 — Pola Segitiga Bilangan

Buat pola bilangan berdasarkan tinggi yang ditentukan pengguna. Tentukan apakah yang perlu ditampilkan adalah counter baris atau kolom dan jelaskan alasannya.

Screenshot source code

```c
#include <stdio.h>

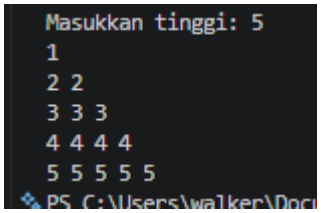
int main(void)
{
    int tinggi;
    int baris;
    int kolom;

    do
    {
        printf("Masukkan tinggi: ");
        scanf("%d", &tinggi);
    } while (tinggi < 1);

    for (baris = 1; baris <= tinggi; baris++)
    {
        for (kolom = 1; kolom <= baris; kolom++)
        {
            printf("%d ", baris);
        }
        printf("\n");
    }

    return 0;
}
```

Screenshot output



Jawab: variabel yang ditampilkan dan jumlah seluruh angka untuk tinggi n

bilangan dari 1 sampai nomor n baris.

3. Level 3 — Tabel Penjumlahan

Buat tabel penjumlahan dari 1 sampai batas. Batas 1–15, gunakan format lebar kolom, nested loop, serta header baris dan kolom.

Screenshot source code

```c
#include <stdio.h>

int main(void)
{
    int batas;
    int baris;
    int kolom;

    do
    {
        printf("Masukkan batas 1-15: ");
        scanf("%d", &batas);
    } while (batas < 1 || batas > 15);

    printf("%5s", "+");

    for (kolom = 1; kolom <= batas; kolom++)
    {
        printf("%5d", kolom);
    }

    printf("\n");

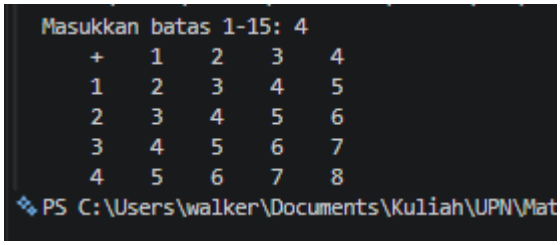
    for (baris = 1; baris <= batas; baris++)
    {
        printf("%5d", baris);

        for (kolom = 1; kolom <= batas; kolom++)
        {
            printf("%5d", baris + kolom);
        }

        printf("\n");
    }

    return 0;
}
```

Screenshot output



4. Challenge — Pola Piramida

Buat pola piramida. Gunakan rumus jumlah spasi = tinggi − baris dan jumlah simbol = 2 × baris − 1. Gunakan dua perulangan dalam: satu untuk spasi dan satu untuk simbol.

Screenshot source code

```c
#include <stdio.h>

int main(void)
{
    int tinggi = 5;
    int baris;
    int spasi;
    int simbol;

    for (baris = 1; baris <= tinggi; baris++)
    {
        for (spasi = 1; spasi <= tinggi - baris; spasi++)
        {
            printf(" ");
        }

        for (simbol = 1; simbol <= 2 * baris - 1; simbol++)
        {
            printf("*");
        }

        printf("\n");
    }

    return 0;
}
```

Screenshot output

```
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemu
    *
   ***
  *****
 *******
*********
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemu
```

Baris	Jumlah Spasi	Jumlah Simbol
1	4	1
2	3	3
3	2	5
4	1	7
5	0	9

5. Bilangan Genap dalam Tabel Perkalian

Buat program latihan_bonus_06.c. Input jumlah baris dan kolom 1–20. Gunakan nested for untuk menghitung jumlah hasil kali genap. Format output harus mengikuti contoh pada modul.

Screenshot source code

bilangan_genap.c

```c
#include <stdio.h>

int main(void)
{
    int n;
    int m;
    int i;
    int j;
    int total_sel = 0;
    int jumlah_genap = 0;

    scanf("%d", &n);
    scanf("%d", &m);

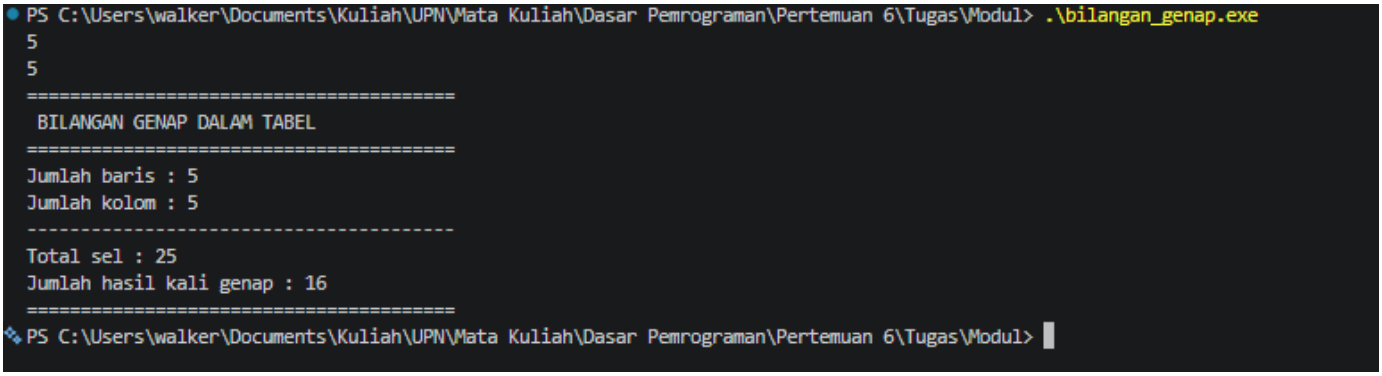
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= m; j++)
        {
            total_sel++;

            if ((i * j) % 2 == 0)
            {
                jumlah_genap++;
            }
        }
    }

    printf("=======================================\n");
    printf(" BILANGAN GENAP DALAM TABEL \n");
    printf("=======================================\n");
    printf("Jumlah baris : %d\n", n);
    printf("Jumlah kolom : %d\n", m);
    printf("---------------------------------------\n");
    printf("Total sel : %d\n", total_sel);
    printf("Jumlah hasil kali genap : %d\n", jumlah_genap);
    printf("=======================================\n");

    return 0;
}
```

Screenshot output



No.	n (baris)	m (kolom)	Expected Bilangan Genap	Actual	Status
1	3	4	8	8	Berhasil
2	5	5	16	16	Berhasil
3	1	10	5	5	Berhasil

VIII. Pengujian Program

1. Pengujian Program Nested Loop

Gunakan program pada VII.1 Level 1 — Persegi Panjang Karakter untuk pengujian berikut. Jalankan setiap test case, catat hasil aktual, lalu tentukan statusnya.

No.	Baris	Kolom	Karakter	Expected Result	Actual Output	Status
1	1	1	*	Satu simbol	*	Berhasil
2	1	5	#	Satu baris, lima simbol	# # # # #	Berhasil
3	5	1	X	Lima baris, satu simbol	X sebanyak 5 baris	Berhasil
4	3	4	+	Tiga baris, empat simbol	++++ sebanyak 3 baris	Berhasil
5	0	4	*	Input baris ditolak	Input ditolak, diminta memasukkan baris 1–20	Berhasil
6	4	21	*	Input kolom ditolak	Input ditolak, diminta memasukkan kolom 1–20	Berhasil

Kesimpulan hasil pengujian

Berdasarkan hasil pengujian, program nested loop dapat menghasilkan pola sesuai dengan jumlah baris, kolom, dan karakter yang diberikan. Input pada batas minimum dan maksimum dapat diproses, sedangkan input yang tidak valid ditolak dan diminta untuk dimasukkan kembali. Semua data uji menghasilkan output sesuai dengan yang diharapkan sehingga program dinyatakan berhasil.

IX. Tugas Praktikum

1. Tugas Praktikum Modul 6 — Program Rekap Penjualan Beberapa Hari dan Produk

Kerjakan tugas utama sesuai spesifikasi pada Modul 6. Source code dikumpulkan sebagai tugas_modul_06_nim.c. Gunakan nested for untuk hari dan produk serta do-while untuk seluruh validasi input.

8. Analisis IPO

Komponen	Uraian
Input	Jumlah hari, jumlah produk, jumlah unit, harga satuan
Process	Validasi input, subtotal produk, total harian, total keseluruhan, jumlah unit, hari tertinggi, rata-rata
Output	Subtotal produk, total harian, total keseluruhan, jumlah unit, rata-rata, hari tertinggi

9. Struktur Perulangan

Perulangan	Jenis	Fungsi
Validasi jumlah hari	Do-while	Validasi 1–7
Validasi jumlah produk	Do-while	Validasi 1–10
Perulangan hari	For	Mengulang setiap hari
Perulangan produk	For	Mengulang produk setiap hari
Validasi jumlah unit	Do-while	Validasi 0–1000
Validasi harga satuan	Do-while	Validasi harga > 0

10. Daftar Accumulator dan Counter

Variabel	Jenis	Diinisialisasi di	Fungsi
total_harian	Accumulator	Awal setiap perulangan hari	Menyimpan total penjualan dalam satu hari
total_keseluruhan	Accumulator	Sebelum perulangan hari	Menyimpan total penjualan dari seluruh hari
seluruh_unit	Accumulator	Sebelum perulangan hari	Menyimpan jumlah seluruh unit yang terjual
hari	Counter	Awal perulangan for hari	Menghitung nomor hari yang sedang diproses
produk	Counter	Awal perulangan for produk	Menghitung nomor produk yang sedang diproses

Pseudocode atau flowchart program

```
// // START

// INPUT jumlah_hari
// WHILE jumlah_hari < 1 OR jumlah_hari > 7
//     INPUT jumlah_hari
// END WHILE

// INPUT jumlah_produk
// WHILE jumlah_produk < 1 OR jumlah_produk > 10
//     INPUT jumlah_produk
// END WHILE

// total_keseluruhan = 0
// seluruh_unit = 0
// nilai_tertinggi = 0
// hari_tertinggi = 0

// FOR hari = 1 TO jumlah_hari
//     total_harian = 0

//     FOR produk = 1 TO jumlah_produk

//         INPUT jumlah_unit
//         WHILE jumlah_unit < 0 OR jumlah_unit > 1000
//             INPUT jumlah_unit
//         END WHILE

//         INPUT harga_satuan
//         WHILE harga_satuan <= 0
//             INPUT harga_satuan
//         END WHILE

//         subtotal = jumlah_unit * harga_satuan
//         total_harian = total_harian + subtotal
//         seluruh_unit = seluruh_unit + jumlah_unit

//     END FOR

//     total_keseluruhan = total_keseluruhan + total_harian

//     IF total_harian > nilai_tertinggi THEN
//         nilai_tertinggi = total_harian
//         hari_tertinggi = hari
//     END IF

// END FOR

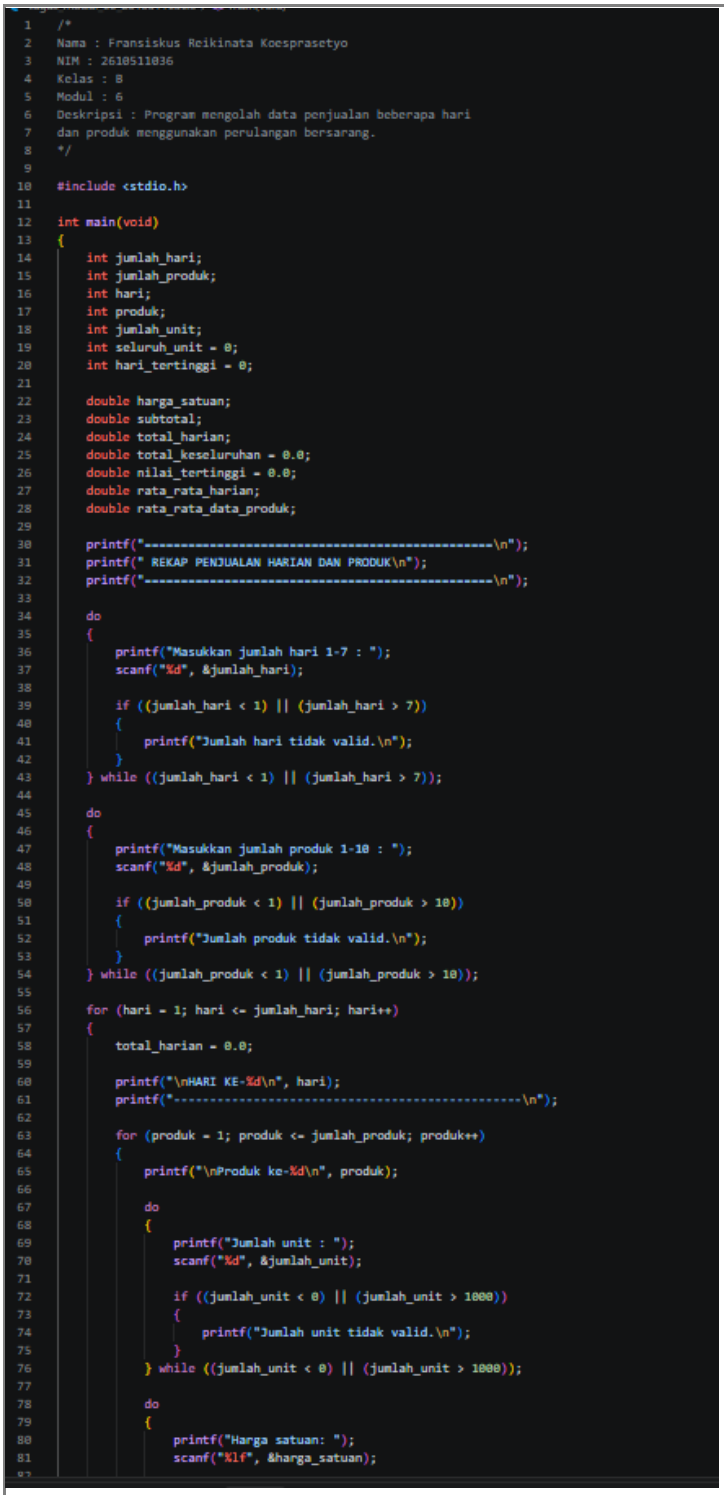
// rata_rata_harian = total_keseluruhan / jumlah_hari
// rata_rata_data_produk = total_keseluruhan / (jumlah_hari * jumlah_produk)

// OUTPUT seluruh_unit
// OUTPUT total_keseluruhan
// OUTPUT rata_rata_harian
// OUTPUT rata_rata_data_produk
// OUTPUT hari_tertinggi
// OUTPUT nilai_tertinggi

// END
```

Trace table untuk satu data uji

Screenshot source code tugas_modul_06_nim.c



```c
/*
Nama : Fransiskus Reikinata Koesprasetyo
NIM : 2610511036
Kelas : B
Modul : 6
Deskripsi : Program mengolah data penjualan beberapa hari
dan produk menggunakan perulangan bersarang.
*/

#include <stdio.h>

int main(void)
{
    int jumlah_hari;
    int jumlah_produk;
    int hari;
    int produk;
    int jumlah_unit;
    int seluruh_unit = 0;
    int hari_tertinggi = 0;

    double harga_satuan;
    double subtotal;
    double total_harian;
    double total_keseluruhan = 0.0;
    double nilai_tertinggi = 0.0;
    double rata_rata_harian;
    double rata_rata_data_produk;

    printf("------------------------------------------------\n");
    printf(" REKAP PENJUALAN HARIAN DAN PRODUK\n");
    printf("------------------------------------------------\n");

    do
    {
        printf("Masukkan jumlah hari 1-7 : ");
        scanf("%d", &jumlah_hari);

        if ((jumlah_hari < 1) || (jumlah_hari > 7))
        {
            printf("Jumlah hari tidak valid.\n");
        }
    } while ((jumlah_hari < 1) || (jumlah_hari > 7));

    do
    {
        printf("Masukkan jumlah produk 1-10 : ");
        scanf("%d", &jumlah_produk);

        if ((jumlah_produk < 1) || (jumlah_produk > 10))
        {
            printf("Jumlah produk tidak valid.\n");
        }
    } while ((jumlah_produk < 1) || (jumlah_produk > 10));

    for (hari = 1; hari <= jumlah_hari; hari++)
    {
        total_harian = 0.0;

        printf("\nHARI KE-%d\n", hari);
        printf("------------------------------------------------\n");

        for (produk = 1; produk <= jumlah_produk; produk++)
        {
            printf("\nProduk ke-%d\n", produk);

            do
            {
                printf("Jumlah unit : ");
                scanf("%d", &jumlah_unit);

                if ((jumlah_unit < 0) || (jumlah_unit > 1000))
                {
                    printf("Jumlah unit tidak valid.\n");
                }
            } while ((jumlah_unit < 0) || (jumlah_unit > 1000));

            do
            {
                printf("Harga satuan: ");
                scanf("%lf", &harga_satuan);
```

Screenshot kompilasi dengan warning aktif (tanpa error/warning penting)

```
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemuan 6\Tugas\Modul> gcc .\tugas_modul_06_2610511036.c -o .\tugas_modul_06_2610511036.exe
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemuan 6\Tugas\Modul> .\tugas_modul_06_2610511036.exe
  ============================================
   REKAP PENJUALAN HARIAN DAN PRODUK
  ============================================
  Masukkan jumlah hari 1-7 :
```

Screenshot hasil eksekusi tugas utama

```
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemuan 6\Tugas\Modul> .\tugas_modul_06_2610511036.exe
Subtotal : Rp60000.00

Total hari ke-2: Rp110000.00

REKAP KESELURUHAN
============================================
Jumlah hari                : 2
Jumlah produk per hari     : 3
Jumlah seluruh unit        : 15
Total keseluruhan          : Rp165000.00
Rata-rata per hari         : Rp82500.00
Rata-rata per data produk  : Rp27500.00
Hari penjualan tertinggi   : Hari ke-2
Nilai tertinggi            : Rp110000.00
============================================
PS C:\Users\walker\Documents\Kuliah\UPN\Mata Kuliah\Dasar Pemrograman\Pertemuan 6\Tugas\Modul>
```

2. Contoh Test Case Tugas

Gunakan minimal delapan test case. Tabel berikut mengikuti contoh pada modul dan dapat digunakan sebagai acuan pengujian.

No.	Hari	Produk	Data Utama	Expected Result	Actual / Status
1	0	–	–	Jumlah hari ditolak	Jumlah hari ditolak/berhasil
2	8	–	–	Jumlah hari ditolak	Jumlah hari ditolak/berhasil
3	1	0	–	Jumlah produk ditolak	Jumlah produk ditolak/berhasil
4	1	11	–	Jumlah produk ditolak	Jumlah produk ditolak/berhasil
5	1	1	Unit 1, harga 10.000	Total 10.000	Total Rp10.000/berhasil
6	1	2	2×5.000 dan 3×10.000	Total 40.000	Total 40.000/ berhasil
7	2	1	Hari 1: 10.000; Hari 2: 20.000	Tertinggi hari 2	Tertinggi hari 2/ Berhasil
8	2	2	Seluruh unit 0	Total 0	Total Rp0/Berhasil
9	2	2	Unit negatif	Input diminta ulang	Input diminta ulang/Berhasil
10	2	2	Harga 0 atau negatif	Input diminta ulang	Input diminta ulang/Berhasil

Catatan debugging selama pengerjaan tugas

Selama pengerjaan, saya menemukan beberapa bagian yang perlu diperiksa, terutama pada penggunaan perulangan bersarang, inisialisasi variabel, dan perhitungan total. Saya memastikan setiap variabel memiliki nilai awal yang sesuai dan memastikan total_harian diatur kembali menjadi 0 pada setiap pergantian hari. Selain itu, saya mengecek kondisi validasi agar input yang berada di luar batas dapat diminta ulang.

Refleksi singkat: bagian tersulit dan cara mengatasinya

Bagian yang paling sulit adalah memahami perulangan bersarang dan cara menghitung total penjualan setiap hari hingga menjadi total keseluruhan. Saya mengatasinya dengan mengikuti alur program secara bertahap, mencoba beberapa data uji, dan mengecek perubahan nilai setiap variabel menggunakan trace table. Dengan cara tersebut, saya lebih mudah mengetahui apakah hasil perhitungan program sudah sesuai dengan yang diharapkan.

3. Diagonal Matriks Perkalian

Buat program tugas_modul_06_soal2_nim.c sesuai modul. Bentuk matriks perkalian n × n dengan nested loop, hitung total diagonal utama, total non-diagonal, dan total keseluruhan.

Screenshot source code tugas_modul_06_soal2_nim.c

tugas_modul_06_soal2_2610511036.c > main(void)

```c
/*
Nama : Fransiskus Reikinata Koesprasetyo
NIM : 2610511036
Kelas : B
Modul : 6
Deskripsi : Program menghitung jumlah nilai diagonal dan non-diagonal
pada matriks perkalian menggunakan nested loop.
*/

#include <stdio.h>

int main(void)
{
    int n;
    int i;
    int j;
    int nilai;
    int total_diagonal = 0;
    int total_non_diagonal = 0;
    int total_keseluruhan = 0;

    printf("=======================================\n");
    printf(" DIAGONAL MATRIKS PERKALIAN\n");
    printf("=======================================\n");

    printf("Ukuran matriks : ");
    scanf("%d", &n);

    printf("---------------------------------------\n");

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            nilai = i * j;

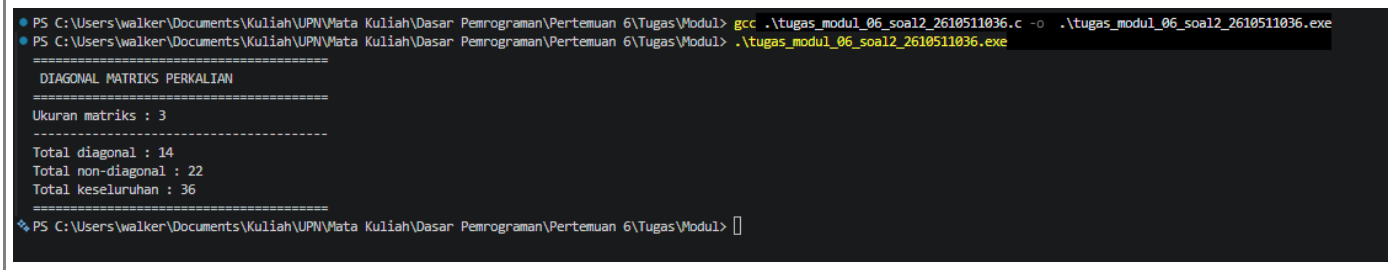
            if (i == j)
            {
                total_diagonal += nilai;
            }
            else
            {
                total_non_diagonal += nilai;
            }
        }
    }

    total_keseluruhan = total_diagonal + total_non_diagonal;

    printf("Total diagonal : %d\n", total_diagonal);
    printf("Total non-diagonal : %d\n", total_non_diagonal);
    printf("Total keseluruhan : %d\n", total_keseluruhan);
    printf("=======================================\n");

    return 0;
}
```

Screenshot output



No.	n	Expected Total Diagonal	Expected Non-diagonal	Expected Total	Actual / Status
1	3	14	22	36	Total diagonal 14, non-diagonal 22, total 36/**Berhasil**
2	4	30	70	100	Total diagonal 30, non-diagonal 70, total 100/**Berhasil**
3	1	1	0	1	Total diagonal 1, non-diagonal 0, total 1/ **Berhasil**

X. Kesalahan Umum dan Standar Penulisan

Pelajari bagian X pada Modul 6 sebelum mengumpulkan LKP. Periksa kembali inisialisasi perulangan dalam, posisi accumulator, lokasi printf("\n"), batas nested loop, arah update counter, penggunaan break, validasi pembagi, dan penggunaan += untuk accumulator.

Pemeriksaan Akhir	Ya/Tidak
Counter perulangan dalam diinisialisasi ulang pada posisi yang benar	Ya
Accumulator per kelompok direset pada awal kelompok	Ya
Accumulator keseluruhan tidak direset di dalam loop luar	Ya
Tidak ada risiko infinite loop	Ya
Indentasi nested loop konsisten	Ya
Program dikompilasi dengan warning aktif	Ya
Tidak ada error atau warning penting	Ya

XI. Pernyataan Penggunaan AI

Pilih salah satu pernyataan berikut dan lengkapi jika diperlukan.

Tidak Menggunakan AI: Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan AI generatif.